Week 1

Intro to C++

For the course:

Object-Oriented Scientific Programming with C++

Version-1

Authors:	Carlos Ene, Mate Kontić, Johannes Nilsson
Style:	TU Delft Report Style, with modifications by Daan Zwaneveld, further modified by Johannes Nilsson for the course WI4771TU/ TW3720TU
Typeface:	Atkinson Hyperlegible/Roboto Slab
Compiler:	PDFTEX

TU Delft

Block A: Estimating π with the Leibniz series

How this block works

In this block you will build a program that estimates π using the Leibniz series. You will not ask the AI to teach you C++. You will ask it to extend your application step by step, the same way an engineer or a scientist would extend a calculation as new questions arise.

After each prompt the AI gives you working code. We will inspect and describe what the C++ code is doing, and how to use the concepts it creates. By the end of the block you will have covered the entire material of Lecture 1 on variables, constants, types and scope, while having built something genuinely useful.

The C++ object we build

The Leibniz formula is the alternating series

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots$$

It converges to $\frac{\pi}{4}$ very slowly, which is a feature for our purposes: it gives us many natural reasons to talk about precision, types, and rounding.

How to work through each step

1. Read the application requirement at the top of the step.
2. Send the prompt to the AI.
3. Look at the code the AI returns
4. Read the “Inside the code” commentary — this is where the C++ concepts are explained.
5. Check the learning goals to confirm you understood.

Step A.1: Get the program to run and announce itself

Before any computation, we need a program that compiles and runs. We want it to print a header line identifying the tool we are about to build, so that we know which version of which calculation we are looking at when the output appears in the terminal.

Prompt

Build a small command-line tool that estimates π using the Leibniz series. For now, just write the program so that it prints a header line: “Leibniz pi estimator — v0.1”. It should compile and run as a complete C++ program.

Generated code:

```cpp
#include <iostream>
int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;
    return 0;
}
```

Inside the Code:

The `#include <iostream>` line pulls in the part of the standard library that provides input and output streams. Without it, `std::cout` would not exist. (Try it yourself. See what error you get.)

The other lines in the code are **statements**, separated by a semicolon `;`. Typically, each statement is written on its own line, though this is for readability, and not strictly required. Each statement is built up of **expressions**, which either have a result, cause a side-effect, or both. For example the expression `5` has the result 5, and so does the expression `2+3`. The expression `x = 5` has the side-effect of setting the variable `x` to 5, and the expression `std::cout << "Hello!"` has the side-effect of printing `hello`

to the terminal. Expressions may consist of other expressions, which are then also evaluated (and may cause their own side effects).

`std::cout` is the standard output stream. The `<<` operator pushes a value into the stream. `std::endl` writes a newline and flushes the buffer, which means the contents of the stream will print on the terminal (the buffer also flushes when your program ends). The `std::` prefix says that `cout` and `endl` belong to the Standard Library's **namespace**, something we will revisit later. If you are familiar with other programming languages (e.g. python), this is equivalent to a `print` statement. The expression `stream << value` not only has the side effect of putting the value in the stream, but also results in `stream`. So `<<` can be chained to push several values. (Since the `<<` operator associates left-to-right)

`main` is the unique entry point of every C++ program. The operating system calls it when the executable starts, and the integer it returns becomes the program's exit code (zero means success). The contents of `main` are wrapped in curly-braces `{}`, this is called a **block**. A block groups statements, creates a scope, and acts as a single statement in control flow.

Learning Goals

- *Structure of a minimal C++ program: header includes plus a main function.*
- *Stream-based output with* `std::cout`*, the* `<<` *operator, and* `std::endl`*.*
- *Why* `std::` *is in front of* `cout`*.*

Step A.2: Output a first, crude estimate

The simplest non-trivial estimate of π from the Leibniz series is obtained by keeping only the first term: $\pi \approx 4$. It is very inaccurate but it gives us something concrete to print and a baseline against which the next steps will improve.

Prompt

Extend the tool so that after the header it prints a first-order estimate of π, obtained by keeping only the first term of the Leibniz series (so $4 \times 1 = 4$). Store the value in a variable so we can build on it later.

Generated code:

```cpp
#include <iostream>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

    double estimate = 4.0 * 1.0;
    std::cout << "Estimate (1 term): " << estimate << std::endl;

    return 0;
}
```

Inside the Code:

`double estimate = 4.0 * 1.0;` introduces a variable. A variable is a reserved part of the computer's memory that represents some number, text, or other data for the program to use. C++ is a typed language: every variable has a type that is fixed for its entire lifetime. The type specifies how a variable works, and how the computer should translate the raw data stored in memory to a useful value. Here the type is `double`, a 64-bit floating-point number (approximately a real number), and the value is initialized on the same line.

The line that introduces the variable does three things. Firstly, it **declares** the variable, telling the compiler that a variable called `double estimate` should exist in the program. This also **defines** the variable, making the computer reserve a portion of memory to store the variable in. For simple variables

this nearly always happens simultaneously. Finally the variable is **initialized** to the value of `4.0`.

Learning Goals

- ***Declaring***, ***defining***, *and* ***initializing*** *a variable in a single line.*
- *C++ is statically typed: each variable has a* ***fixed type****.*
- *Chaining multiple values into a single* `std::cout` *statement.*

Step A.3: Add the second term

We now include the second term of the series, $1-1/3$, so the estimate becomes $4\times(1-1/3) = 8/3 \approx 2.667$. This is a more useful estimate than the previous one, but it brings up a numerical subtlety that is easy to miss.

Prompt

Now refine the estimate by including the second term of the series: $\pi \approx 4 \times (1 - \frac{1}{3})$. Compute it using two separate variables for the two terms, then print the new estimate next to the previous one so we can see the improvement. Use integer variables for the two terms, since the indices of the series are integers.

Generated code:

```cpp
#include <iostream>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

    int t1 = 1;
    int t2 = 1 / 3;
    double estimate = 4.0 * (t1 - t2);

    std::cout << "Estimate (1 term):  " << 4.0 << std::endl;
    std::cout << "Estimate (2 terms): " << estimate << std::endl;

    return 0;
}
```

Running the program prints `4` for the second estimate as well, which is clearly incorrect: we expected `2.667`. Something has gone wrong.

Inside the Code:

The bug is in `int t2 = 1 / 3;`. Because both operands are `int`, the compiler performs integer division, which truncates towards zero: `1 / 3` evaluates to `0`, not `0.333`. The variable `t2` holds zero, and the estimate ends up being $4 \times (1 - 0) = 4$.

This is a common mistake in C++, and a dangerous one since it happens silently. To fix it, we can replace the integers with floating point numbers instead.

Tip

When the AI produces a result you did not expect, resist the urge to immediately ask for a fix. First ask the AI to explain what is happening — for example: "Why is the second estimate equal to 4?". Understanding the failure is more valuable than getting the corrected code.

Prompt

Replace the integers used to calculate the second term with doubles

Generated code:

```cpp
#include <iostream>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

    int t1 = 1;
    double t2 = 1.0 / 3.0;
    double estimate = 4.0 * (t1 - t2);

    std::cout << "Estimate (1 term):  " << 4.0      << std::endl;
    std::cout << "Estimate (2 terms): " << estimate << std::endl;

    return 0;
}
```

Now `t1` and `t2` are double-precision floating-point numbers, the division `1.0 / 3.0` is computed with floating-point arithmetic, and the estimate prints as approximately `2.66667`.

The literals `1.0` and `3.0` are not the same things as `1` and `3`. The decimal point tells the compiler to interpret them as `double` values. These are called suffix-free floating-point literals. If we wanted a single precision `float` instead, we could write `1.0f`.

Tip

It's fine to mix `int` and `double` in calculations. When C++ encounters values of different types, it promotes the values to the "biggest" type, and does the arithmetic using the promoted value. The inverse happens if you try to assign a `double` value to an `int` variable, the value is truncated to the "smaller" type.

Learning Goals

- *C++ has multiple numeric types; `int` for integers, `double` for double precision floating point, and `float` for single precision.*
- *integer division truncates.*
- *Literals also carry a type: `1` is an `int`, `1.0` is a `double`, `1.0f` is a `float`.*
- *The type of an expression is determined by the types of its operands.*
- *mixed arithmetic promotes values*

Step A.4: Report the error against a known value of π

To check the accuracy of our tool, it should compare its estimate against a high-precision reference value of π and report the absolute error. The reference value is a physical constant of our calculation: it should be fixed for the entire lifetime of the program.

Prompt

Add a high-precision reference value of π (`3.14159265358979`) to the program and print the absolute error of each estimate against this reference. The reference value must not be modifiable anywhere in the code

Generated code:

```cpp
#include <iostream>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

```

```
    const double pi_ref = 3.14159265358979;

    double t1 = 1.0;
    double t2 = 1.0 / 3.0;

    double est_1 = 4.0;
    double est_2 = 4.0 * (t1 - t2);

    std::cout << "Reference pi:        " << pi_ref << std::endl;
    std::cout << "Estimate (1 term):  " << est_1
              << "    error: " << pi_ref - est_1 << std::endl;
    std::cout << "Estimate (2 terms): " << est_2
              << "    error: " << pi_ref - est_2 << std::endl;

    return 0;
}
```

Inside the Code:
`const double pi_ref = 3.14159265358979;` introduces a constant. The `const` keyword tells the compiler that the value is fixed when defined and may not be reassigned. Writing `pi_ref = 3.14;` later in the program would cause a compilation error, which is exactly the guarantee we asked for.

Two language rules follow from this. First, a `const` must be initialized on the line that defines it: writing `const double pi_ref;` on its own is illegal, because the compiler would never get another chance to give the constant a value. Second, the `const` protection is enforced at compile time, not at run time, there is no runtime `const` variable.

Learning Goals

- *The `const` qualifier prevents modification after initialization.*
- *A `const` must be initialised on the same line where it is defined.*
- ***Declaration** vs. definition: a declaration introduces a name, a definition allocates storage.*

Step A.5: Replace the hard-coded sum with a loop
Up to now the sum has been written out by hand. That obviously does not scale: if we want a hundred terms, or a thousand, we cannot keep typing them. We want the program to automatically compute terms up to some number `N` To do that, we need a loop that walks through the terms one by one and adds them up with the correct alternating sign.

Prompt

Rewrite the program so that the estimate is computed by a for loop that runs from `k = 0` to `k = N - 1` and adds each term `(-1)^k / (2k + 1)` to a running sum, which is then multiplied by `4`. Use an `int` for the loop index `k`. Be careful that the division inside the term is done in floating-point arithmetic, not integer arithmetic. Set `N` to `1000` so we can see the estimate get closer to the reference.

Generated code:

```
#include <iostream>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

    const double pi_ref = 3.14159265358979;
    const int    N      = 1000;     // number of terms used in the
   estimate

```

```
9       double sum = 0.0;
10      for (int k = 0; k < N; ++k) {
11          const double sign = (k % 2 == 0) ? 1.0 : -1.0;
12          const double term = sign / (2.0 * k + 1.0);
13          sum += term;
14      }
15      const double estimate = 4.0 * sum;
16
17      std::cout << "Using N = " << N << " terms" << std::endl;
18      std::cout << "Estimate: " << estimate
19                << "    error: " << pi_ref - estimate << std::endl;
20
21      return 0;
22  }
```

Inside the Code:
The `for` loop has three parts inside its parentheses. The **initialization** `int k = 0;` declares the loop index and runs once, before the loop starts. The **condition** `k < N` is checked at the beginning of every iteration; as soon as it becomes `false` the loop ends. The **update** `++k` runs at the end of every iteration after the body, incrementing `k` by one. Together these three pieces produce exactly `N` iterations, with `k` taking the values `0`, `1`, `2`, ..., `N - 1`. (C++ also has the `while` loop, which only has the condition part)

Note that `k` is declared inside the loop header. That puts it in the **scope** of the loop block. `k` is only scoped from line 9 to line 14. Once the loop ends, `k` no longer exists. This is true for all variables. They only exist inside the scope they are declared in: `sign`, `term`, and `k` only exist inside the loop, and cannot be accessed outside it. `pi_ref`, `N`, `sum`, and `estimate` are in the scope of `main`, and cannot be accessed outside of it. However since the loop is also in `main`, it's fine to access `sum` inside the loop.

Inside the body, the sign of each term alternates. We compute it with the ternary expression `(k % 2 == 0) ? 1.0 : -1.0`, which evaluates to `1.0` when `k` is even and `-1.0` when `k` is odd. The denominator `2.0 * k + 1.0` mixes the `int k` with the `double` literals `2.0` and `1.0`, so `k`'s value is automatically promoted to `double` and the whole expression is computed in floating-point. If we had written `2 * k + 1` instead, the denominator would have been an `int`, and dividing `1.0` (a `double`) by it would still have worked because of the same promotion rule, but writing the literals as `2.0` and `1.0` makes the intent obvious to a reader.

The variable sum is the running total. We initialize it to `0.0` (a `double`, not an `int`, otherwise `sum += term` would try to add a `double` to an `int`) and the `+=` operator adds the right-hand side to it on every iteration. After the loop finishes, `sum` holds `1 - 1/3 + 1/5 - 1/7 + ...` up to `N` terms, and multiplying by `4` gives the estimate of π.

Tip

When you ask the AI for a loop, be explicit about the index range: say "from 0 to N - 1" or "from 1 to N", not just "loop N times". C++ uses zero-based indexing almost everywhere, but the AI will happily give you a 1-based loop if your prompt is ambiguous, and an off-by-one error in a sum is exactly the kind of silent bug we saw in Step A.3.

Learning Goals

- *The structure of a `for` loop: initialization, condition, update.*
- *Scope: an index declared in the loop header or inside the loop block lives only inside the loop.*
- *the Modulo operator `%`, the addition assignment operator `+=`, and the ternary expression condition `Q ? A : B`*

Step A.6: Refuse bad input by throwing an exception
The loop only makes sense for `N > 1`. If someone sets `N` to zero, the loop body never runs and the estimate is `0`, the same kind of failure mode we already met with integer division in Step A.3. If someone sets `N` to a negative number, the situation is even more embarrassing: the condition `k < N` is false from the start and we again get `0`, with no warning. We want the program to refuse to run in those cases instead of producing a meaningless answer.

The right tool for this in C++ is an exception:

Prompt

Before the loop starts, check whether `N` is at least `1`. If it is not, signal an error using the C++ exception mechanism: throw a `std::invalid_argument` with a message that explains what went wrong. Include the `<stdexcept>` header. Do not catch it yet — we will do that in the next step. For now we just want the program to refuse to compute an estimate when `N` is not positive.

Generated code:

```cpp
#include <iostream>
#include <stdexcept>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

    const double pi_ref = 3.14159265358979;
    const int    N      = 0;

    if (N < 1) {
        throw std::invalid_argument(
            "N must be at least 1 (got a non-positive value).");
    }

    double sum = 0.0;
    for (int k = 0; k < N; ++k) {
        const double sign = (k % 2 == 0) ? 1.0 : -1.0;
        const double term = sign / (2.0 * k + 1.0);
        sum += term;
    }
    const double estimate = 4.0 * sum;

    std::cout << "Using N = " << N << " terms" << std::endl;
    std::cout << "Estimate: " << estimate
              << "   error: " << pi_ref - estimate << std::endl;

    return 0;
}
```

Inside the Code:

The new include line, `#include <stdexcept>`, brings in the standard exception types defined by the C++ Standard Library. `std::invalid_argument` is one of them, and it is the type the C++ standard recommends when a function (or in our case `main`) is given an argument that does not satisfy its requirements.

The line `throw std::invalid_argument("...")` does two things in one go. It constructs a temporary `std::invalid_argument` object, passing the message string to its constructor, and then it throws that object. From that point on, the normal flow of the program stops.

What happens next, with no catch block anywhere is: the runtime calls `std::terminate()`, which aborts the program with a message like:
"terminate called after throwing an instance of '`std::invalid_argument`'".

Tip

Exceptions are for exceptional conditions, not for normal flow control. "N is negative" qualifies — it should never happen in a correctly used program. "the user pressed q to quit", on the other hand, is normal flow and should be handled with a plain `if`. If you ask the AI to "use exceptions" for ordinary branching, it will comply, and your code will become harder to read and slower to run.

Learning Goals

- *Including `<stdexcept>` to get access to the standard exception types.*
- *Throwing an exception with `throw std::invalid_argument(message)`.*
- *What the standard exception hierarchy looks like (`invalid_argument`, `out_of_range`, `runtime_error`, ...) and that all of them derive from `std::exception`.*
- *An uncaught exception terminates the program; a clean program catches what it throws.*
- *Validate inputs early, before they can corrupt the rest of the calculation.*

Step A.7: Catch the exception and fail gracefully
An uncaught exception terminates the program with a runtime message that is fine for a developer but ugly for a user. A scientific tool should fail gracefully: print a clear error message, exit with a non-zero status code so a calling script knows something went wrong.

Prompt

Put the calculation inside a try block and add a catch (`const std::exception& e`) block that prints the error message returned by `e.what()` to `std::cerr` and makes main return a non-zero exit code. Catch by `const` reference, not by value, and catch `std::exception` (the common base class), not `std::invalid_argument` specifically — that way the same handler will work if we later throw a different exception type for a different kind of problem.

Generated code:

```
#include <iostream>
#include <stdexcept>

int main() {
    std::cout << "Leibniz pi estimator -- v0.1" << std::endl;

    const double pi_ref = 3.14159265358979;
    const int    N      = 0;

    try {
        if (N < 1) {
            throw std::invalid_argument(
                "N must be at least 1 (got a non-positive value).");
          }

        double sum = 0.0;
        for (int k = 0; k < N; ++k) {
            const double sign = (k % 2 == 0) ? 1.0 : -1.0;
            const double term = sign / (2.0 * k + 1.0);
            sum += term;
          }
```

```
22          const double estimate = 4.0 * sum;
23
24          std::cout << "Using N = " << N << " terms" << std::endl;
25          std::cout << "Estimate: " << estimate
26                    << "   error: " << pi_ref - estimate << std::endl;
27        }
28      catch (const std::exception& e) {
29          std::cerr << "Error: " << e.what() << std::endl;
30          return 1;
31      }
32
33      return 0;
34  }
```

Inside the Code:
The `try { ...}` block marks a region of code in which an exception might be thrown. If any function call inside it throws an exception, the program halts normal execution and jumps to the matching `catch` block. The matching is done by type: `catch (const std::exception& e)` accepts any exception whose type is `std::exception` or any class that derives from it.

Two details of the `catch` line are required for good C++ style code. First, the parameter should be a reference (`&`), not a value. If we catch (`std::exception e`) by value, the exception object would be **sliced**, that is, copied into a plain `std::exception` and stripped of any extra information the derived class carried. Catching by reference keeps the full object intact. Second, it should be `const`, because we are only going to read the exception's message, never modify it; marking it `const` documents that intent and prevents accidental writes.

Try the program with `N = 0`, `N = -5`, and `N = 1000` and watch what happens. For the first two, the throw fires, the catch handler prints a clean `"Error: N must be at least 1..."` message to standard error, and the program exits with code `1`. For `N = 1000`, the `if` condition is `false`, the throw is skipped, the loop runs, and the estimate prints as before.

Tip

The general rule of thumb in C++ error handling is: `throw` by value, `catch` by `const` reference, and catch the most specific type that you actually need to handle. We caught `std::exception` here because we only have one kind of error to report and we just want to print its message. Once a program has several distinct failure modes — bad input, file not found, network timeout — it pays to write several `catch` blocks, each handling its own type.

Learning Goals

- *Wrapping risky code in a `try` block and pairing it with a `catch` block.*
- *Why we catch by `const` reference (avoids object slicing, documents read-only intent).*
- *Catching the base class `std::exception` to handle any derived exception with one handler.*
- *Using `e.what()` to read the message stored in the exception.*
- *Printing to `std::cerr` rather than `std::cout`, and returning a non-zero exit code on failure.*

What you built, and what you learned
You started with a one-line print statement and ended with a small, scientifically meaningful tool that estimates π, reports its error against a known reference, exposes a configurable number of terms, and is written in a clean modern style.

Along the way you introduced several C++ concepts, such as streams, variables, integer and floating point types, loops, and exceptions.

This is the pattern the rest of the course will follow: Consider a scientific problem while introducing new

C++ features that help solve it.

Block B: Refactoring the estimator into functions and namespaces

How this block works

In this block you will continue the Leibniz π estimator from Block A and turn it from a single fat main function into a properly structured C++ program.

The C++ object we build

The Leibniz formula is the alternating series

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots$$

It converges to $\frac{\pi}{4}$ very slowly, which is a feature for our purposes: it gives us many natural reasons to talk about precision, types, and rounding.

Step B.1: Extract the sum into a function

At the end of Block A, `main` was doing four different things: declaring the parameters of the calculation, validating them, running the loop, and printing the result. That is too many responsibilities for a single function. The first cleanup is to extract the loop into its own function with a clear name, so that someone reading `main` can see what happens without having to read how it happens.

We want a function `leibniz_estimate` that takes the number of terms `N` and returns the estimate of π. The input validation is part of the function's contract — "I refuse to run if `N` is not at least `1`" — so it moves inside the function as well. `main` keeps the `try` / `catch` around the call site, where the result of the failure has to be handled.

Prompt

Define a function called `leibniz_estimate` that takes one integer parameter (the number of terms `N`) and returns a `double` — the estimated value of π. Move the loop and the input check from `main` into this function. Keep main responsible only for setting up `N`, calling the function inside a `try` block, printing the result, and handling exceptions. The function should be defined before `main` so we don't need a separate declaration.

Generated code:

```cpp
#include <iostream>
#include <stdexcept>

double leibniz_estimate(int N) {
    if (N < 1) {
        throw std::invalid_argument(
            "N must be at least 1 (got a non-positive value).");
    }

    double sum = 0.0;
    for (int k = 0; k < N; ++k) {
        const double sign = (k % 2 == 0) ? 1.0 : -1.0;
        sum += sign / (2.0 * k + 1.0);
    }
    return 4.0 * sum;
}

int main() {
    std::cout << "Leibniz pi estimator -- v0.2" << std::endl;

```

```
    const int N = 1000;

    try {
        const double estimate = leibniz_estimate(N);
        std::cout << "Using N = " << N << " terms" << std::endl;
        std::cout << "Estimate: " << estimate << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Inside the Code:
The parameter `N` is passed **by value**. That means at the moment the function is called, the `int` value in the argument is copied into the local variable `N` inside the function. If the function modified `N`, say with `N = 0;` only the function's local copy would change; `main`'s `N` would still be `1000`.

Notice where the function lives in the file: above `main`. This matters because the compiler needs to know what `leibniz_estimate` is before it sees the call to it inside `main`. If the function was below `main` the compiler would have no idea what "`leibniz_estimate`" was when it reads it in `main`.

If you still want to have the function below `main`, you can declare the function without defining its body by only writing its signature: `double leibniz_estimate(int N);` above `main`. This is enough information for the compiler to use it. Even if it doesn't know exactly what the function does, it knows how it behaves.

Tip

When you ask the AI to "refactor this into a function", be specific about the signature: "a function that takes `int N` and returns `double`". Otherwise it might invent parameters you did not ask for, or pick a return type you did not want. The signature is the contract between caller and function; pin it down explicitly.

Learning Goals

- *Defining a function with a return type, name, parameter list, and body.*
- *Pass by value: the function receives a copy of the argument; modifying the parameter does not affect the caller.*
- *Function declaration vs. function definition*
- *Using `return` to send a value of the declared type back to the caller.*

Step B.2: Report the error through a reference parameter
Right now the function gives us the estimate, but to know whether the estimate is any good we have to compare it to a reference value of π back in `main`. The function should compute the error itself and report it back.

A C++ function returns only one value directly. We are already using this return value for the estimate. The classic way to give a function a second output is to take a parameter **by reference** and have the function write into it.

Prompt

Extend `leibniz_estimate` *so that it also reports the absolute error of its estimate against the reference value of* π*. The reference value of* π *should live as a* `const double` *inside the function (*`3.14159265358979`*). Add a second parameter,* `double& error`*, that the function fills in with the absolute difference between the estimate and the reference. Include* `<cmath>` *for* `std::abs`*. In* `main`*, declare a* `double err`*; before the call and print it after.*

Generated code:

```cpp
#include <iostream>
#include <stdexcept>
#include <cmath>

double leibniz_estimate(int N, double& error) {
    if (N < 1) {
        throw std::invalid_argument(
            "N must be at least 1 (got a non-positive value).");
    }

    const double pi_ref = 3.14159265358979;

    double sum = 0.0;
    for (int k = 0; k < N; ++k) {
        const double sign = (k % 2 == 0) ? 1.0 : -1.0;
        sum += sign / (2.0 * k + 1.0);
    }
    const double estimate = 4.0 * sum;

    error = std::abs(pi_ref - estimate);
    return estimate;
}

int main() {
    std::cout << "Leibniz pi estimator -- v0.2" << std::endl;

    const int N = 1000;

    try {
        double err;
        const double estimate = leibniz_estimate(N, err);
        std::cout << "Using N = " << N << " terms" << std::endl;
        std::cout << "Estimate: " << estimate << std::endl;
        std::cout << "Error:    " << err << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Inside the Code:

The new parameter `double& error` is a reference, as marked by the ampersand `&`. When the function is called, `error` does not become a copy of the caller's variable, it becomes another name for the caller's variable. The two names refer to the same storage. Writing `error = 1` inside the function is the same

as writing `err = 1` outside, because they both refer to the same place in memory.

On the caller's side, the syntax does not change at all: we just write `leibniz_estimate(N, err)`. The compiler binds the reference parameter to our variable automatically.

Inside the function, `error = std::abs(pi_ref - estimate);` computes the absolute value of the difference. `std::abs` comes from the `<cmath>` header, alongside other mathematical functions. We use the absolute value because we care about the magnitude of the error, not its sign.

Tip

Returning a second value through a reference parameter is the classic, pre-modern way to do it in C++. It works, but it has a downside: the function's outputs are split between the return value and the argument list, which makes the call site less obvious to read. Later in the course we will meet cleaner alternatives — returning a `struct`, or using `std::pair` / `std::tuple` — that keep all outputs in the return value.

Learning Goals

- *Pass by reference: the parameter is another name for the caller's variable, not a copy.*
- *The function can write through a reference parameter and the caller sees the change.*
- *Why the call site looks identical whether the parameter is by value or by reference — the difference is in the function's signature.*
- *Including `<cmath>` to get access to `std::abs` and other mathematical functions.*
- *Why returning a second value via reference, while valid, is something we will eventually replace with cleaner alternatives.*

Step B.3: Add a read-only printer with const references

`main` is now short, but the three `std::cout` lines still feel out of place there: printing is a separate concern from computing the estimate. The third extraction is a small `print_report` function that takes the numbers we computed and writes them out in a consistent format.

This function only reads its inputs, it never writes to them. We want the compiler to enforce that intent, both as documentation for the reader and as a safety net against accidentally modifying something we did not mean to. The right tool is the `const` reference: `const T&`.

Prompt

Add a function `print_report` that prints the value of `N`, the estimate, and the error to `std::cout`, with one labelled line each. It returns nothing (use `void` as the return type). Pass all three parameters by `const` reference: `const int& N, const double& estimate, const double& error`.

Generated code:

```
#include <iostream>
#include <stdexcept>
#include <cmath>

double leibniz_estimate(int N, double& error) {
    if (N < 1) {
        throw std::invalid_argument(
            "N must be at least 1 (got a non-positive value).");
    }

    const double pi_ref = 3.14159265358979;

    double sum = 0.0;
```

```
    for (int k = 0; k < N; ++k) {
        const double sign = (k % 2 == 0) ? 1.0 : -1.0;
        sum += sign / (2.0 * k + 1.0);
    }
    const double estimate = 4.0 * sum;

    error = std::abs(pi_ref - estimate);
    return estimate;
}

void print_report(const int& N,
                  const double& estimate,
                  const double& error) {
    std::cout << "Using N = " << N << " terms" << std::endl;
    std::cout << "Estimate: " << estimate     << std::endl;
    std::cout << "Error:    " << error        << std::endl;
}

int main() {
    std::cout << "Leibniz pi estimator -- v0.2" << std::endl;

    const int N = 1000;

    try {
        double err;
        const double estimate = leibniz_estimate(N, err);
        print_report(N, estimate, err);
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Inside the Code:
`const T&` is still pass by reference: the function sees the caller's variable without copying it, but the `const` qualifier prevents any modification — if we accidentally wrote `estimate = 0.0;` inside `print_report`, the compiler would complain. The `const` reference also ensures the caller that the variable won't be changed, even though it's passed by reference. The benefit over passing by value is that there's no need to duplicate the variable to use inside the function. This isn't really a benefit for basic types like `int` or `double`, but for larger objects (vectors, matrices, strings) this can save both memory and time.

The `void` return type signals that the function returns nothing. Its impact on the program is instead some side-effect like printing a value.

Look at main now. It is six lines: declare `N`, open the `try` block, get the `estimate` and `error`, hand them to `print_report`, close the block, handle exceptions. Every C++ concept beyond the very first is now hidden in helper function.

Tip

The rule of thumb to internalize is: pass-by-value for parameters the function will modify locally without affecting the caller, by `const` *reference for read-only inputs, and by plain reference for out-parameters the function writes to. Following this convention makes every function's intent visible from its signature alone.*

Learning Goals

- *Pass by* `const` *reference: read-only access to the caller's variable, with the compiler enforcing the read-only promise.*
- *The* `void` *return type and what it means for the function body.*
- *Separating orchestration (in* `main`*) from computation and reporting (in named functions).*
- *Reading a function signature is enough to understand its contract: who can modify what.*

Step B.4: Group the functions in a namespace

The two helper functions have generic names: `leibniz_estimate` is specific enough, but `print_report` could clash with another library that also has a `print_report`. As programs grow and include third-party code, name collisions become a real problem. C++ solves this with **namespaces**, which are named regions in which a set of names lives.

You have been using one namespace already: `std`. Every Standard Library feature — `cout`, `endl`, `abs`, `invalid_argument`, `exception` — lives in `namespace std`, and we reach them with `std::name`. We will do the same with our own functions: put them in `namespace leibniz` and call them as `leibniz::estimate` and `leibniz::print_report`.

Prompt

Wrap both `leibniz_estimate` *and* `print_report` *in a namespace called* `leibniz`*. Inside the namespace, rename* `leibniz_estimate` *to just* `estimate`*. Update* `main` *to call* `leibniz::estimate(N, err)` *and* `leibniz::print_report(N, estimate, err)`*. Do not use a using directive — keep the* `leibniz::` *prefix at every call site so the reader can see where each name comes from.*

Generated code:

```
#include <iostream>
#include <stdexcept>
#include <cmath>

namespace leibniz {

    double estimate(int N, double& error) {
        if (N < 1) {
            throw std::invalid_argument(
                "N must be at least 1 (got a non-positive value).");
        }

        const double pi_ref = 3.14159265358979;

        double sum = 0.0;
        for (int k = 0; k < N; ++k) {
            const double sign = (k % 2 == 0) ? 1.0 : -1.0;
            sum += sign / (2.0 * k + 1.0);
        }
        const double est = 4.0 * sum;

```

```
        error = std::abs(pi_ref - est);
        return est;
    }

    void print_report(const int& N,
                      const double& estimate,
                      const double& error) {
        std::cout << "Using N = " << N << " terms" << std::endl;
        std::cout << "Estimate: " << estimate     << std::endl;
        std::cout << "Error:    " << error        << std::endl;
    }

}  // namespace leibniz

int main() {
    std::cout << "Leibniz pi estimator -- v0.2" << std::endl;

    const int N = 1000;

    try {
        double err;
        const double est = leibniz::estimate(N, err);
        leibniz::print_report(N, est, err);
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Inside the Code:

`namespace leibniz { ... }` creates a new named region. Everything declared inside it becomes part of that region and must be referred to as `leibniz::name` from outside. The closing brace ends the `namespace`. Inside the `namespace`, names refer to each other without the prefix. If `print_report` wanted to call `estimate`, it could just write `estimate(N, err)`. From outside the `namespace`, both must be qualified.

We could also have written `using namespace leibniz;` at the top of `main` to introduce the contents of `leibniz` into the `main` scope. This would've allowed us to access them just by their own name. This may seem convenient, but defeats the point of namespaces and is a bad habit that should be used sparingly.

Tip

When you ask the AI to "put this code in a `namespace`", be explicit about whether you want a `using` directive at the call site or not. AI will often add "`using namespace leibniz;`" for convenience and quietly undo the namespacing benefit.

Learning Goals

- *Defining a named* `namespace` *to group related functions and prevent name clashes.*
- *Calling a namespaced function from outside its namespace using the* `namespace::name` *form.*
- *Inside a* `namespace`*, sibling names can be referred to without qualification.*
- *Why "*`using namespace X;`*" (and especially "*`using namespace std;`*") is discouraged in serious C++ code.*

Step B.5: Tighten the contracts with assertions

The function already throws on `N < 1`, which is the right behaviour for a runtime check: the caller might have `N` coming from user input, a configuration file, or a network message, and the program needs to handle that without crashing. But there is another kind of check we have not used yet: the kind that says "this should *never* happen if I wrote the code correctly".

For example: by the time we reach the line `error = std::abs(pi_ref - estimate);`, the variable `sum` should hold a finite number. If `sum` is somehow `NaN` or `Inf`, something has gone catastrophically wrong in the loop and continuing is pointless. We do not want a runtime exception for that since it is not the user's fault, it is the programmer's, and the right reaction is to stop immediately so the bug can be found. The tool for this is `assert`.

Prompt

Inside the estimate function, after the loop has finished and before the error is computed, add an assert that the running sum is a finite number (use `std::isfinite` *from* `<cmath>`*, which is true for normal floating-point values and false for* `NaN` *and infinity). Also add an assert at the top of* `print_report` *that* `N` *is at least* `1` *— a sanity check that we are not being called with garbage. Keep the existing* `throw` *for* `N < 1` *in estimate; the assert is a developer check, not a replacement for the runtime check.*

Generated code:

```cpp
#include <iostream>
#include <stdexcept>
#include <cmath>
#include <cassert>

namespace leibniz {

    double estimate(int N, double& error) {
        if (N < 1) {
            throw std::invalid_argument(
                "N must be at least 1 (got a non-positive value).");
        }

        const double pi_ref = 3.14159265358979;

        double sum = 0.0;
        for (int k = 0; k < N; ++k) {
            const double sign = (k % 2 == 0) ? 1.0 : -1.0;
            sum += sign / (2.0 * k + 1.0);
        }
        assert(std::isfinite(sum));
        const double est = 4.0 * sum;

        error = std::abs(pi_ref - est);
        return est;
    }
```

```
27
28      void print_report(const int& N,
29                        const double& estimate,
30                        const double& error) {
31          assert(N >= 1);                    // developer check
32          std::cout << "Using N = " << N << " terms" << std::endl;
33          std::cout << "Estimate: " << estimate     << std::endl;
34          std::cout << "Error:    " << error        << std::endl;
35      }
36
37  }  // namespace leibniz
38
39  int main() {
40      std::cout << "Leibniz pi estimator -- v0.2" << std::endl;
41      const int N = 1000;
42
43      try {
44          double err;
45          const double est = leibniz::estimate(N, err);
46          leibniz::print_report(N, est, err);
47      }
48      catch (const std::exception& e) {
49          std::cerr << "Error: " << e.what() << std::endl;
50          return 1;
51      }
52
53      return 0;
54  }
```

Inside the Code:

`assert` is a **macro** that comes from the C library. Its argument is a boolean expression. If the expression is `true`, nothing happens. If it is `false`, the program aborts immediately with a message like "Assertion '`std::isfinite(sum)`' failed" that includes the file, line, and condition. It is deliberately abrupt: an assertion failure means a bug in the program, and continuing in a broken state would make the bug harder to diagnose.

The difference between `assert` and `throw` is what the user sees when something goes wrong. A `throw` produces a clean, catchable exception with a message the program can present to the user; the program continues running and can recover. An `assert` produces an abort and a core dump; the program is over. The distinction is by intent: throw is for predictable failures the caller is expected to handle, `assert` is for impossible situations that signal a programming error.

There is one more important detail. When you compile a program with optimisation flags for release (typically by defining the macro `NDEBUG`), the `assert` macro expands to nothing — the check disappears entirely from the executable. This is by design: assertions are development-time safety nets, and you do not want them slowing down release builds. The consequence is critical: never put an assert around an expression with side effects, like `assert(file.open())` — in a release build the file would never open. The condition must be a pure check, like `assert(N >= 1)` or `assert(std::isfinite(sum))`.

Tip

When you ask the AI to "add validation" to a function, it will tend to reach for `throw` by default. Be explicit about the kind of check you want: "add a runtime check (`throw`)" for user-facing failures, "add a developer assertion (`assert`)" for "should-never-happen" cases. The AI will happily do either, but without that distinction it usually mixes them up.

Learning Goals

- *How* `assert(condition)` *behaves: silent if* `true`*, abort with a diagnostic if* `false`*.*
- *The semantic difference between* `throw` *(runtime failure, catchable, user-visible) and* `assert` *(programmer error, fatal, development-time only).*
- *Why assertions disappear in release builds (*`NDEBUG`*) and the consequence: never put side effects inside an assert.*
- *Usage of* `std::isfinite` *from* `<cmath>` *to test that a floating-point value is a regular number (not NaN, not infinity).*

What you built, and what you learned

Block A produced a tool that worked, but with everything in one `main` function. Block B turned it into a small, properly structured C++ program: the calculation, the reporting, and the orchestration are now three separate, named things, grouped under their own **namespace**, with developer-time and runtime checks each doing the job they are meant to do.

`main` is now six lines of orchestration and nothing more. Every computational concept lives in a well-named function, every function is in a `namespace`, and the boundary between programmer error and runtime error is made explicit. The next block will introduce classes, which are a blueprint for designing scientific objects yourself.

Block C: Arrays, initialisation, and pointers

How this block works

In this block you will build a small statistics tool from scratch: an array of measurements, with functions that compute the mean and the variance. By the end of the block you will have covered the last remaining material of Lecture 1: the four initialization forms, arrays, and C++ pointers and their hazards.

The object we build

Given a sample of N measurements $x_0, x_1, \ldots, x_{N-1}$, we want to compute two basic statistics: the sample mean $\overline{x}$ and the sample variance σ^2, defined as:

$$\overline{x} = \frac{1}{N}\sum_i x_i$$

$$\sigma^2 = \frac{1}{N-1}\sum_i (x_i - \overline{x})^2$$

From the variance we get the standard deviation σ as its square root

Step C.1: Store the sample in a fixed-size array

Before computing any statistics, we need somewhere for the measurements to live. A natural choice in C++ is a fixed-size **array**: a contiguous list of variables, declared with a known length, holding the values one after another in memory. We will write the values directly into the source code for now and just print them, to confirm that we can store and read back the data.

Prompt

Write a C++ program that declares an array of 5 doubles called `data`*, initialized in place with the values* `2.1, 2.3, 1.9, 2.2, 2.0` *using brace-list initialization. Then loop over the array with a* `for` *loop and print each element to* `std::cout`*, one per line, prefixed with its index (e.g.* `data[0] = 2.1`*). Use plain* `int` *for the loop index. The array length should be a* `const int n = 5;` *declared just above the array.*

Generated code:

```
#include <iostream>

```

```cpp
int main() {
    std::cout << "Sample statistics tool -- v0.1" << std::endl;

    const int n = 5;
    double data[n] = {2.1, 2.3, 1.9, 2.2, 2.0};

    for (int i = 0; i < n; ++i) {
        std::cout << "data[" << i << "] = " << data[i] << std::endl;
    }

    return 0;
}
```

Inside the Code:

The line `double data[n] = {2.1, 2.3, 1.9, 2.2, 2.0};` declares and initializes an array. The type is `double[n]` (an array of doubles) and the compiler reserves enough contiguous memory for five doubles, one after another. The braces on the right are a **brace-enclosed initializer list**: the first value goes into `data[0]`, the second into `data[1]`, and so on. If the list has fewer values than the array length, the remaining elements are zero-initialized; if it has more, the compiler raises an error.

`data[i]` is the subscript operator. It accesses the `i`-th element of the array, counting from zero. We used a `const int n = 5;` for the length, then declared the array with that name. Note that this only works with `n` being `const`. Since its a static array, the size needs to be known at compile time. We'll get to how to allocate arrays of dynamic size later.

Tip

When you ask the AI to declare an array of fixed values, say "use brace-list initialization" explicitly. Otherwise it may give you data[0] = 2.1; data[1] = 2.3; ... five times in a row (which is correct but less concise and clear)

Learning Goals

- *Declaring a fixed-size C-style array:* `double data[n]`.
- ***Brace-list initialization*** *of an array: the first list value goes into* `[0]`*, the next into* `[1]`*, and so on.*
- *Accessing an element with the subscript operator* `data[i]`.
- *Using a* `const int` *for the length so the array size lives in exactly one place in the source code.*

Step C.2: Compute the mean and meet the four initialisation forms

Now that we have the data, we can compute the sample mean. What makes this step interesting is the four different ways C++ lets us declare and initialise the local variables we need.

C++ inherits two initialization styles from C (the assignment (`=`) form and the parentheses form), adds one of its own in C++11 (the curly-brace form), and also allows a variable to be declared with no initialiser at all, which — as we will see — is a small trap waiting to happen. This step uses every form deliberately so we can compare them.

Prompt

Extend the program to compute and print the mean of the array. Inside `main`*, deliberately use four different initialization forms for the local variables: declare* `sum` *with copy-style "*`double sum = 0.0;`*", declare* `count` *with constructor-style "*`int count(n);`*", declare the result* `mean` *with empty-brace value-initialization "*`double mean{};`*" and declare a loop index* `i` *with no initializer at all ("*`int i;`*"). Then loop, add up the values, and compute the* `mean = sum / count`*. Print the* `mean` *on its own labeled line. Add a code comment next to each declaration explaining which initialization form it is.*

Generated code:

```cpp
#include <iostream>

int main() {
    std::cout << "Sample statistics tool -- v0.1" << std::endl;

    const int n = 5;
    double data[n] = {2.1, 2.3, 1.9, 2.2, 2.0};

    double sum   = 0.0;     // copy initialisation
    int    count(n);        // direct (constructor) initialisation
    double mean{};          // value initialisation (zero for built-in
   types)
    int    i;               // no initialiser -- holds an arbitrary
   value!

    for (i = 0; i < count; ++i) {
        sum += data[i];
    }
    mean = sum / count;

    std::cout << "Mean = " << mean << std::endl;

    return 0;
}
```

Inside the Code:

Copy initialization: `double sum = 0.0;`. The `=` looks like an assignment, but at the point of declaration it is actually initialization.

Direct (constructor) initialization: `int count(n);`. The parentheses say "construct the variable directly from this value". For built-in types like `int` the result is identical to the `=` form, but for class types (which we will meet later in the course) the parentheses call the matching constructor.

Value initialization with empty braces: `double mean{};`. The empty braces zero-initialize built-in types: `mean` is born holding `0.0`. The braced form, introduced in C++11, has two important advantages over the `=` and `( )` forms. First, it is uniform: it works the same way for built-in types, class types, and arrays. Second, it forbids narrowing conversions — for example, `int x{3.7};` refuses to compile (because `3.7` cannot be exactly represented in an `int`), whereas `int x = 3.7;` silently truncates to `3`. This makes brace-init the safest default in modern C++ when you want a variable to start at a known value.

No initializer: `int i;`. For a local variable of a built-in type, this leaves `i` holding whatever bits happened to be in that memory location, an unpredictable value. Reading `i` before assigning to it is **undefined behavior**. In our code this is safe because the first thing we do is `i = 0;` in the for header. But it is a common source of bugs in larger programs, and the cure is simple: when in doubt, use brace-init with no value (`T name{};`) which guarantees zero for built-in types. The general rule of thumb is: never leave a variable uninitialized unless you have a specific reason.

Tip

If you ask the AI to "initialize a variable to zero", it will typically pick `= 0.0` *because that is the most common form in training data. To get brace-init you have to say so explicitly: "use brace value-initialization,* `double x{};` *". This is the only way to build the habit of writing* `T x{};` *instead of* `T x = 0;` *in your own code.*

Learning Goals

- *Copy initialization:* `T name = value;` *(the form we used in Blocks A and B).*
- *Direct (constructor) initialization:* `T name(value);` *— identical to* `=` *for built-in types, but able to call multi-argument constructors for class types.*
- *Value initialization with empty braces:* `T name{};` *— zero-initializes built-in types, forbids narrowing conversions, works uniformly across types.*
- *The trap of declaring a local variable with no initializer: it holds arbitrary bits; reading it before assigning is undefined behavior.*
- *Brace-init (*`T name{value};`*) as the safest modern default, especially because it refuses silent narrowing conversions.*

Step C.3: Extract the mean into a function and meet pointers

Computing the mean is a self-contained idea: take a sequence of numbers, return their average. It deserves its own function, exactly as in Block B. But there is a complication: a C-style array, when passed to a function, does not arrive as an array. It arrives as a **pointer** to its first element, and the function has no idea how long it is.

All variables in the program have an **address**, corresponding to where the bits that represent their value are stored in the computer's memory. **Pointers** point to an address, and allow us to share it around in our program. Two operators, the address-of operator `&` and the **dereference** operator `*`, allow use to work with pointers. `int *p;` creates a pointer `p` that points at integers, `p = &x;` assigns the address of `x` to the pointer `p`, `std::cout << *p;` **dereferences** the pointer `p` and prints the result to the terminal. Once `p` points to the variable `x`, the expression `*p` is equivalent to `x`. The reason this is useful is that passing around the address of `x` allows us to change it from elsewhere in the program. Passing `x` into a function only gives us the value, whatever we do inside the function has no effect on the `x` outside the function. But if we pass the address, then we can directly access the memory of `x`, and change it inside the function, similar to passing a varaible as a reference.

Pointers are also how we access arrays. Since arrays are contiguous in memory, the address of the i-th member of the array is i steps from the start of the array. If we have the address of `data` as a pointer `double *p = &data[0]`, then the 5th element of the array can be accessed by `*(p+4)`. (This is why we need to specify *what* the pointer points to. Different types of variables have different sizes in memory, and the pointer must translate "`p+2`" to the right offset.)

The "array" variable we've been using before is in fact equivalent to a pointer to the first element. The expressions `data[i]` and `*(data+i)` are equivalent. As are `data` and `&data[0]`.

Prompt

Extract the `mean` *calculation into a function called* `mean` *that takes two parameters: a pointer to a* `double` *(*`double* data`*) and the length of the array (*`int n`*). The function returns the mean as a* `double`*. Place the function above* `main`*. Inside the function, use the subscript form* `data[i]` *in the loop.*

Generated code:

```
#include <iostream>

double mean(double* data, int n) {
    double sum{};
```

```
    for (int i = 0; i < n; ++i) {
        sum += data[i];           // equivalent to *(data + i)
    }
    return sum / n;
}

int main() {
    std::cout << "Sample statistics tool -- v0.2" << std::endl;

    const int n = 5;
    double data[n] = {2.1, 2.3, 1.9, 2.2, 2.0};

    const double m = mean(data, n);
    std::cout << "Mean = " << m << std::endl;

    return 0;
}
```

Inside the Code:

Why do we have to pass `n` separately? Because the function only receives a pointer to the first element. It has no way of knowing where the array ends. This is the central weakness of C-style arrays and the reason `std::vector` and `std::array` (which carry their length with them) are preferred in modern code. We will meet them later in the course.

Pointers and the address-of operator are the C++ machinery for talking about memory locations directly, and we will see them again in Step C.5 when we deal with bad pointers.

Tip

When you ask the AI to "refactor this to use a function that takes an array", be explicit that you want C-style pointer-and-length parameters: "the function signature should be `double mean(double* data, int n)` *". Otherwise the AI is likely to use* `std::vector` *or a C++20* `std::span` *— both excellent in their place, but neither is what Lecture 1 is teaching. You will meet those later.*

Learning Goals

- *A pointer (`T*`) is a variable that stores the address of another variable.*
- *array-to-pointer decay: passing an array to a function turns it into a pointer to its first element; the length information is lost and must be passed separately.*
- *The equivalence* `data[i]` = `*(data + i)` *: subscript is shorthand for dereference of pointer arithmetic.*
- *The address-of operator* `&` *gives the address of a variable;* `*p` *dereferences a pointer to access what it points to.*
- *Why C-style arrays "force" every function to take a length parameter.*

Step C.4: Add the variance and the standard deviation

With the mean in place, the variance is one more loop: subtract the mean from each value, square the difference, sum the squares, and divide by the count. The function signature is almost the same as mean's, with one extra parameter — the precomputed mean — so we do not have to compute it twice.

Prompt

Add a function called `variance` *that takes the same* `double* data` *and* `int n` *as mean, plus a third parameter* `double mean_value` *(the precomputed mean). The function returns the sample variance as a* `double`*. In* `main`*, after computing the mean, call* `variance(data, n, mean_value)` *to get the variance, then compute the standard deviation as* `std::sqrt(variance_value)` *using* `<cmath>`*. Print all three numbers on their own labelled lines.*

Generated code:

```cpp
#include <iostream>
#include <cmath>

double mean(double* data, int n) {
    double sum{};
    for (int i = 0; i < n; ++i) {
        sum += data[i];
    }
    return sum / n;
}

double variance(double* data, int n, double mean_value) {
    double sum_sq{};
    for (int i = 0; i < n; ++i) {
        double diff = data[i] - mean_value;
        sum_sq += diff * diff;
    }
    return sum_sq / (n-1);
}

int main() {
    std::cout << "Sample statistics tool -- v0.2" << std::endl;

    const int n = 5;
    double data[n] = {2.1, 2.3, 1.9, 2.2, 2.0};

    const double mean_value   = mean(data, n);
    const double var = variance(data, n, mean_value);
    const double sd  = std::sqrt(var);

    std::cout << "Mean   = " << mean_value   << std::endl;
    std::cout << "Var    = " << var << std::endl;
    std::cout << "StdDev = " << sd  << std::endl;

    return 0;
}
```

Inside the Code:

`variance` has the same interface as `mean`: same pointer-and-length pattern, same loop structure, same return type. Only the body differs, and instead of accumulating `data[i]`, it accumulates `(data[i] - mean_value)` squared.

`variance` taking in mean as a parameter saves us the time of computing it twice, once in each function. However this means that the caller has to remember to pass this value in every time they call the function.

Tip

When you ask the AI to add a second function that operates on the same array, specify that it should use the same pointer-and-length convention as the first. Otherwise you may get a mix — one function takes `(double*, int)`*, the next takes* `std::vector<double>`*, the next takes some third obscure format — and the call site becomes a museum of every C++ feature for passing arrays. Pick one style per file and stick to it.*

Learning Goals

- *Why a derived quantity (like a precomputed mean) is passed in rather than recomputed — and the price you pay: the caller must keep the inputs consistent.*
- *Using* `std::sqrt` *from* `<cmath>`*, which is overloaded for the floating-point types.*

Step C.5: Guard the functions against pointer hazards

There are three flavours of bugs associated with pointer hazards, and any of them turns a one-line slip into a program that crashes or, worse, silently produces wrong answers.

The first hazard is the **uninitialized pointer**: a pointer declared with no initialiser, pointing to some random part of memory. Dereferencing it will cause undefined behaviour, and if the memory address doesn't belong to your program, it causes a segmentation-fault and crashes your program.

The second possible pointer hazard is a **null pointer**. Null pointers are pointers that hold the address "nowhere", meaning this pointer points to nothing. They are used because they help make bugs easier to detect, the statement `if(p)` is always `false` when `p = nullptr` (for normal pointers it is always `true`), so setting all invalid pointers to `nullptr` allows you to check before dereferencing. If you forget the check and still dereference them it's still undefined behaviour, so you still need to be careful.

The third is the **dangling pointer**: a pointer to memory that has been freed or has gone out of scope. In effect causing the same issue as if it was pointing to a random address.

All three produce undefined behaviour when dereferenced. In this step we will guard explicitly against the first, and use the discussion to explain how to avoid the other two.

Prompt

Add an input check at the top of both `mean` *and* `variance`*: if the data pointer is* `nullptr`*, or if* `n` *is less than* `1`*, throw* `std::invalid_argument` *with a clear message. In* `main`*, wrap the three function calls in a* `try` *block and add a matching* `catch` *(*`const std::exception& e`*) that prints the message to* `std::cerr` *and returns* `1`*. Use the keyword* `nullptr` *explicitly (not* `NULL`*, not* `0`*) when comparing the pointer.*

Generated code:

```
#include <iostream>
#include <cmath>
#include <stdexcept>

double mean(double* data, int n) {
    if (data == nullptr) {
        throw std::invalid_argument("mean: data pointer is null.");
    }
    if (n < 1) {
        throw std::invalid_argument("mean: n must be at least 1.");
    }

    double sum{};
    for (int i = 0; i < n; ++i) {
        sum += data[i];
```

```
    }
    return sum / n;
}

double variance(double* data, int n, double mean_value) {
    if (data == nullptr) {
        throw std::invalid_argument("variance: data pointer is null.");
    }
    if (n < 1) {
        throw std::invalid_argument("variance: n must be at least 1.");
    }

    double sum_sq{};
    for (int i = 0; i < n; ++i) {
        const double diff = data[i] - mean_value;
        sum_sq += diff * diff;
    }
    return sum_sq / (n-1);
}

int main() {
    std::cout << "Sample statistics tool -- v0.3" << std::endl;

    const int n = 5;
    double data[n] = {2.1, 2.3, 1.9, 2.2, 2.0};

    try {
        const double m   = mean(data, n);
        const double var = variance(data, n, m);
        const double sd  = std::sqrt(var);

        std::cout << "Mean   = " << m   << std::endl;
        std::cout << "Var    = " << var << std::endl;
        std::cout << "StdDev = " << sd  << std::endl;
    }
    catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Inside the Code:

`nullptr` is the C++11 keyword for the null pointer. It is a distinct value of type `std::nullptr_t` that converts to any pointer type as the null pointer of that type. Before C++11 the same idea was spelled `NULL` (a macro defined as `0`) or just `0` — both of which had the awkward property of being integer values in the wrong context, leading to overload-resolution bugs (e.g. `f(NULL)` calling `f(int)` instead of `f(void*)`). `nullptr` fixes that: it is genuinely a pointer value, not a number that happens to be zero, and modern code uses it exclusively.

The check `if (data == nullptr)` is the safety check. If a caller uses a `nullptr` the function refuses immediately rather than dereferencing a null pointer (which on most systems crashes the program with a segmentation fault).

Learning Goals

- *The three pointer hazards: null, uninitialised, and dangling — what each one is, and what kind of bug each one produces.*
- *Using* `nullptr` *(the C++11 keyword) instead of* `NULL` *or* `0` *for the null pointer value.*
- *Guarding a function against a null pointer argument by an explicit check and a throw — the same pattern as the input validation in Block A.*
- *Initialising pointers with* `T* p{};` *to make them start as* `nullptr`*, so the same check protects against the uninitialised-pointer case.*
- *Why references are safer than pointers when the parameter must refer to a real object, and why pointers are still the right tool when null is a meaningful value.*

What you built, and what you learned

Block C built a small statistics tool from scratch: an array of measurements, two functions that compute the mean and the variance, the standard deviation derived in main, and explicit guards against invalid inputs

You used arrays to store multiple variables, passed them to functions using pointers, learned about the four initialization forms and the three pointer hazards.

Exercises

You have now learned the basics of C++ and constructed a few C++ programs. To get a clearer understanding, try to solve some of these exercises. First by yourself, and if you run into problems, alongside an LLM.

1. The `<cmath>` header includes a lot of mathematical functions, one it doesn't include is the *factorial* $n! = 1 \cdot 2 \cdot \ldots \cdot (n-1) \cdot n$. Create a function to compute the factorial of a number.
   (Hint: $n! = n \cdot (n-1)!$ and $0! = 1$)

2. Alongside π, the other famous transcendental number is e. Modify the Leibniz program to instead compute e using the series:

$$\mathrm{e} = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + 1 + \frac{1}{2} + \frac{1}{2 \cdot 3} + \frac{1}{2 \cdot 3 \cdot 4} + \ldots$$

3. We created a program to compute the mean and variance of a set of values. Use the same format to compute a) the norm, and b) the median of the values.

4. Just as `std::cout <<` can print values to the terminal, `std::cin >>` can read values from the terminal. Modify the mean calculation program to take in the values from the user.
   (Hint: when reading from the terminal, `(std::cin >> x)` is `true` if the read was successful, and `false` otherwise.)

5. Since we have not yet introduced a way to create dynamically sized arrays, exercise 4 will have a maximum number of elements the user can enter. Therefore, modify the program to calculate the mean and variance *without* using an array.

Alphabetical Index

array, 19

block, 2
brace-list initialization, 20

compiler, 1

declaration, 2
double, 2

exception, 7
expression, 1

header, 27

initialization, 2
integer, 2

macro, 18

namespace, 2

pointer, 19

signature, 11
statement, 25

terminal, 1
ternary expression, 6

undefined behavior, 21